CS 486/686
Dissecting a Vision-Language Model

Yuntian Deng

Lecture 23

How a language model sees

Before we start: end-of-term updates

EXTRA OFFICE HOURS

Wed Jul 29 · 3–4 PM

DC 2633

Thu Jul 30 · 4–5 PM

MC 4021

FINAL · SAT AUG 8

7:30–10:00 PM · PAC 5

Bring a non-programmable calculator + one double-sided letter-size notes sheet.

Two sample exams are on LEARN.

REVIEW MATERIALS

A1 + A2 solutions are on LEARN

Assignment 2 grades have also been released.

UPCOMING DEADLINES

A3 · Tue Aug 4

Final project · Wed Aug 5

Both at 11:59 PM Waterloo time.

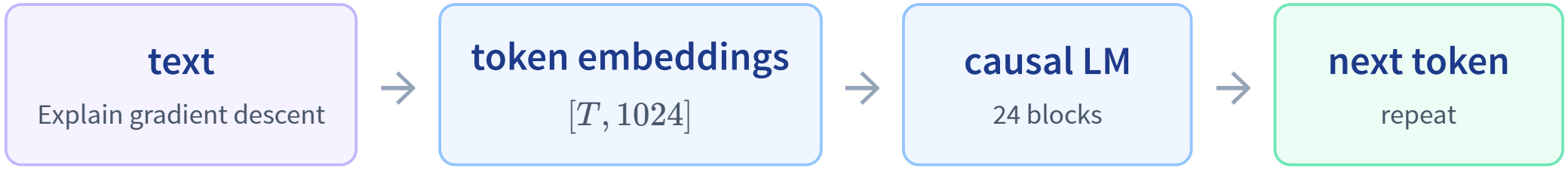
Full details stay current on the course page.

Search › Uncertainty › Decisions › Learning

By the end, you can trace how Qwen sees

1. **Run** visual Q&A and OCR in your browser.
2. **Trace** pixels → patches → image embeddings.
3. **Calculate** how resolution changes the token count.
4. **Explain** how a familiar embedding interface lets us reuse an LLM.

Without an image, Qwen is the LM you already know



L21 already explained this path. Today we change the input, not the output loop.

The exact model we will dissect

Qwen3.5-0.8B

post-trained checkpoint

`Qwen/Qwen3.5-0.8B`

1024
LM hidden width

24
language blocks

16×16
image patches

text
output modality

Qwen Team, “Qwen3.5: Towards Native Multimodal Agents,” 2026; official model configuration.

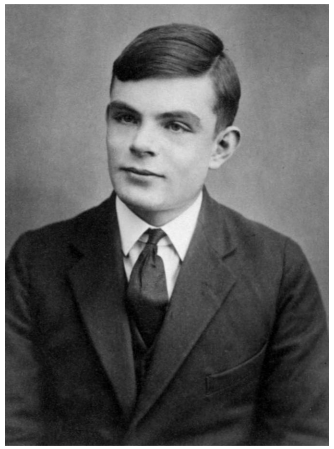
One twist: most blocks carry a recurrent state

Long image contexts do not pay full quadratic attention in every block.

Ask Qwen about an image LIVE IN BROWSER

Choose the photo—or upload your own—then ask a visual question.

Slow-device warning: live generation can take around 20 minutes. The recorded result is immediate, and you can stop live generation at any time.



RECORDED QWEN3.5-0.8B OUTPUT

QUESTION

Is the person wearing a suit? What are they wearing?

ANSWER

Yes, the person is wearing a suit. They are dressed in a dark suit jacket, a white collared shirt, and a dark tie.

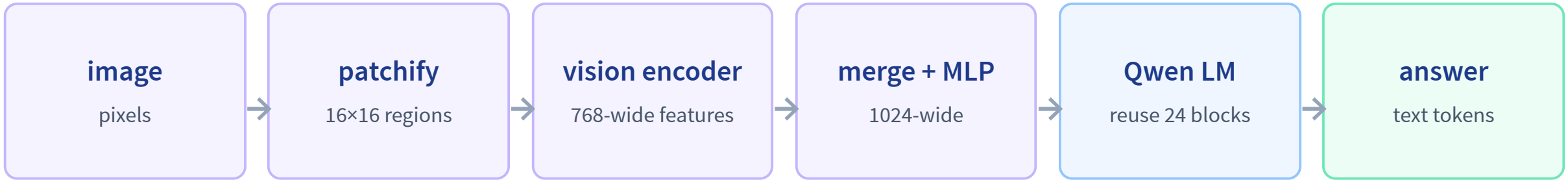
Live path: Transformers.js v4 + WebGPU + onnx-community/Qwen3.5-0.8B-ONNX. No API key; inference stays in the browser.

THE CENTRAL QUESTION

The language model did not receive pixels.

What vectors did it receive?

Make the image look like language to the LLM



Pixels and token IDs begin in different worlds

image

`[384, 640, 3]`

737,280 RGB numbers

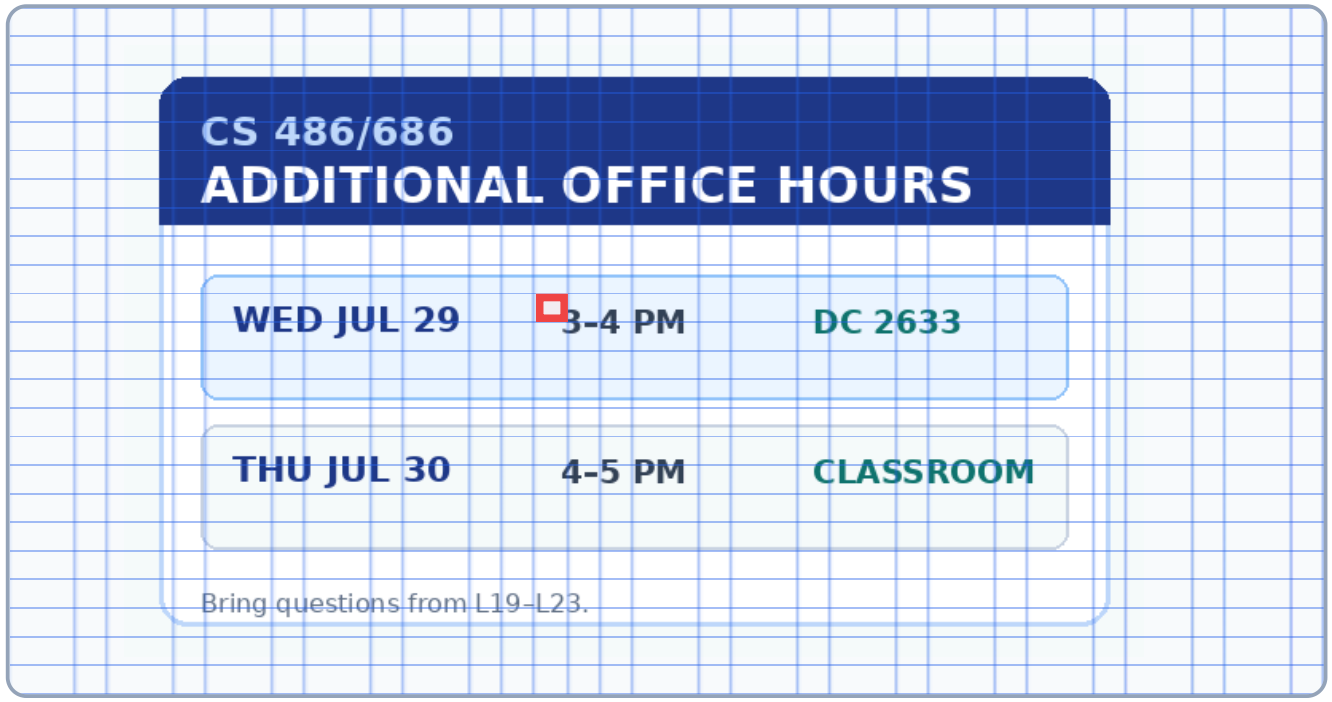
text

`[2488, 374, 2201, …]`

a short sequence of IDs

First compress local pixels into a sequence of patch vectors.

Qwen starts with 16×16 image patches



one patch
$16 \times 16 \times 3 = 768$ pixel values

one learned vector
the patch embedding

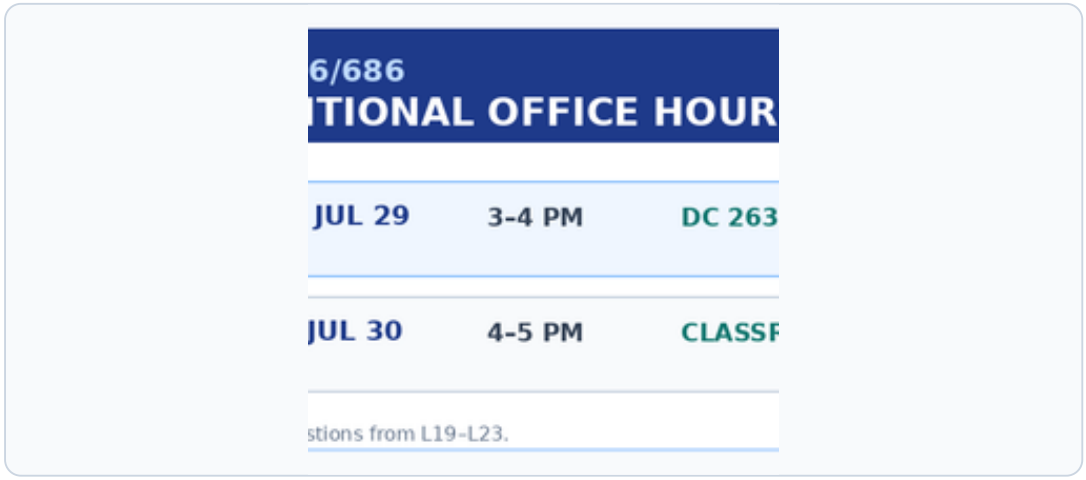
all patches
a spatial sequence

Patchify while preserving the image shape LIVE

Change the retained resolution; count raw patches and merged image embeddings.

640×384 image → 40×24 raw patches → 20×12 = **240 image embeddings**

A fixed square resize can erase small details



original CLIP-style input
224×224 square

variable-resolution input
640×384, aspect preserved

Documents need enough retained patches for their text to survive.

“Native resolution” means a variable patch grid

preserve
aspect ratio

round
dimensions to patch-compatible sizes

cap
the total visual-token budget

The processor does not promise to retain every original pixel;
it chooses the largest useful grid within its configured budget.

SigLIP 2 NaFlex and Qwen multimodal image preprocessing use variable grids to reduce aspect-ratio distortion.

More retained pixels create a longer visual sequence

256×256

16×16 = 256 raw patches

64 after 2×2 merge

512×512

32×32 = 1024 raw patches

256 after 2×2 merge

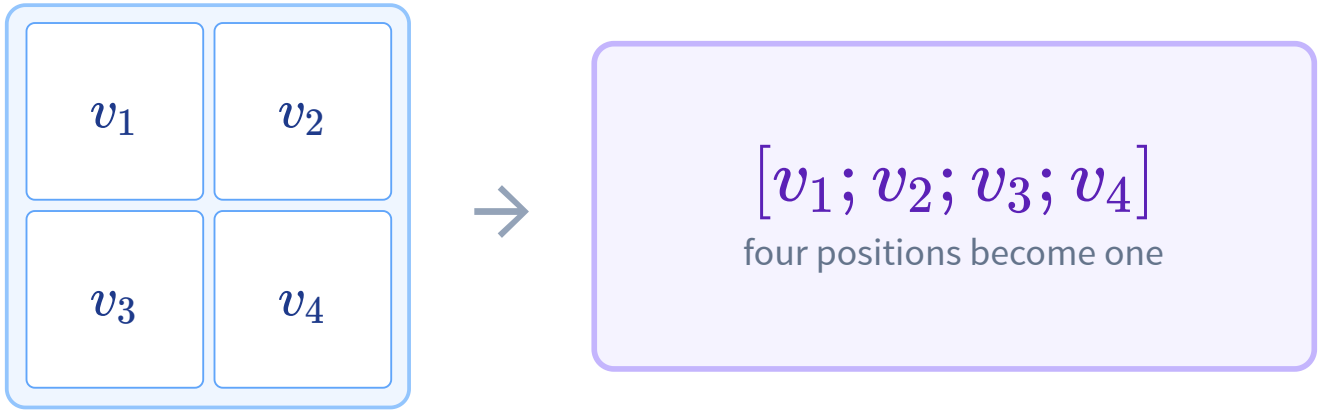
640×384

40×24 = 960 raw patches

240 after 2×2 merge

Resolution buys detail by spending context and compute.

Qwen merges each 2×2 neighborhood



$$N_{\text{LM image}} = \frac{H}{16}\frac{W}{16}\frac{1}{4} = \frac{H}{32}\frac{W}{32}$$

PATCHES ARE ONLY THE START

How does a patch come to mean “cup” or “deadline”?

It must be trained to carry visual semantics.

Recall CLIP: put matching images and text nearby

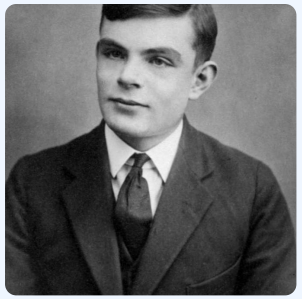
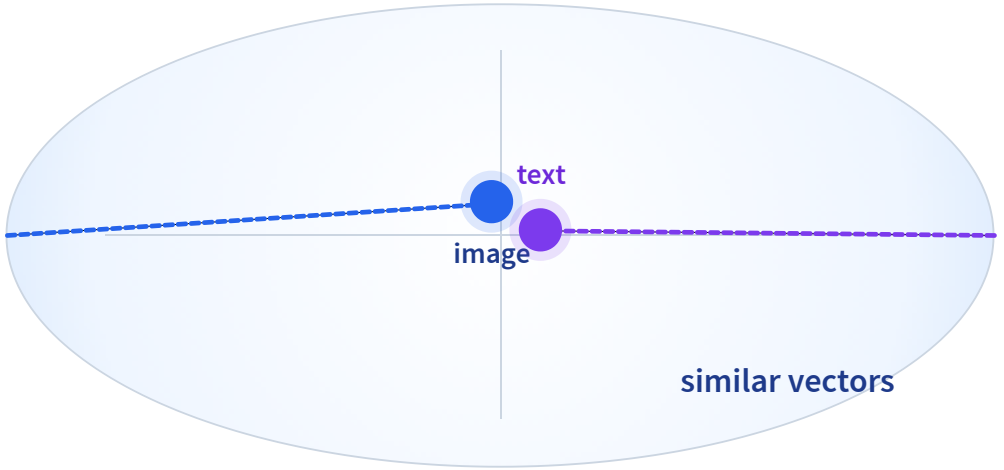


image encoder

$f_I(I)$ → blue point



“a person wearing a suit”

text encoder

purple point ← $f_T(T)$

Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” 2021. Callback to L18.

Each image finds its caption—and vice versa LIVE

The diagonal contains real pairs; other cells are in-batch mismatches.

photo ↔ “person”	photo ↔ “chart”
chart ↔ “person”	**chart ↔ “chart”**

Contrastive training turns appearance into meaning

before

colors, edges, textures

after matching text

objects, actions, concepts

Now the vision features are useful to a language model—not merely compact pixels.

Qwen’s vision tower is derived from SigLIP2

CLIP
contrastive image–text alignment

→

SigLIP2
stronger multilingual and dense visual features

→

Qwen tower
adapted vision encoder + Qwen merger

We use the idea and move on; the SigLIP2 training refinements are optional reading.

Tschannen et al., “SigLIP 2,” 2025; Qwen3.5 vision configuration.

The vision tower contextualizes every patch

patch embeddings

↓

12 vision blocks

↓

$[N, 768]$ contextual features

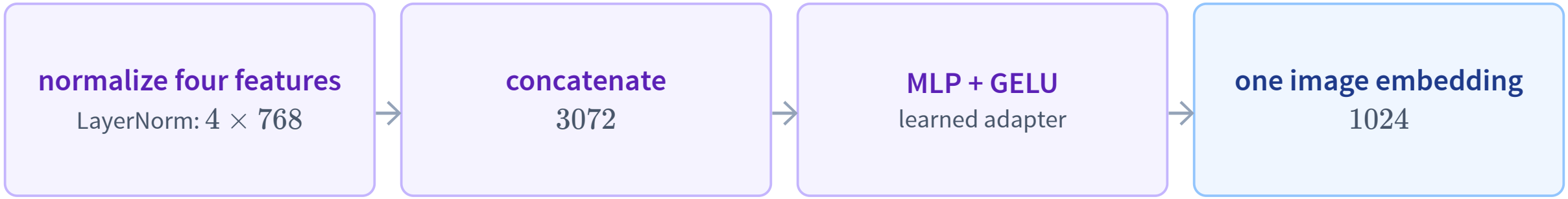
HIDDEN WIDTH
768

ATTENTION HEADS
12

PATCH SIZE
16×16

The same tower also handles video with temporal patches; today we follow one still image.

The connector changes count and width



Qwen3.5 visual merger: LayerNorm, 2×2 spatial merge, two linear layers with GELU.

Same shape means the same LLM interface

image embeddings

`[V, 1024]`

from the connector

word embeddings

`[T, 1024]`

from the lookup table

```
llm(inputs_embeds = concat(image_embeds, text_embeds))
```

To the backbone, both are sequences of 1024-dimensional vectors.

Turn the unknown into the known

This pattern appears everywhere in computer science: adapt, reduce, reuse.

The image and question become one context

Image spans can appear wherever the prompt places them.
After embedding, every position uses the same 1024-D interface.

Once embedded, generation is unchanged

multimodal prefix
image + question embeddings

→

hybrid Qwen LM
same 24 blocks

→

next text token
append and repeat

Vision changes prefill; decoding still grows the text answer one token at a time.

End-to-end training teaches Qwen to use the adapter

Where are Wednesday's office hours?

They are in **DC 2633**.

vision tower ← connector ← Qwen LM ← next-token loss

Early-fusion multimodal training makes the connected system useful—not the connector shape alone.

One image, every boundary

The recorded trace is generated from the real processor;
the demo reports its actual grid for uploaded images.

The complete browser path is short

```
const processor = await AutoProcessor.from_pretrained(MODEL);
const model = await Qwen3_5ForConditionalGeneration
  .from_pretrained(MODEL, {
    device: "webgpu",
    dtype: {
      embed_tokens: "q4",
      vision_encoder: "fp16",
      decoder_model_merged: "q4",
    },
  });

const prompt = processor.apply_chat_template(messages, {
  add_generation_prompt: true,
  enable_thinking: false,
});
const inputs = await processor(prompt, [image]);
const output = await model.generate({ ...inputs, max_new_tokens: 64 });
```

processor
patch + serialize

model
encode + adapt + reuse LM

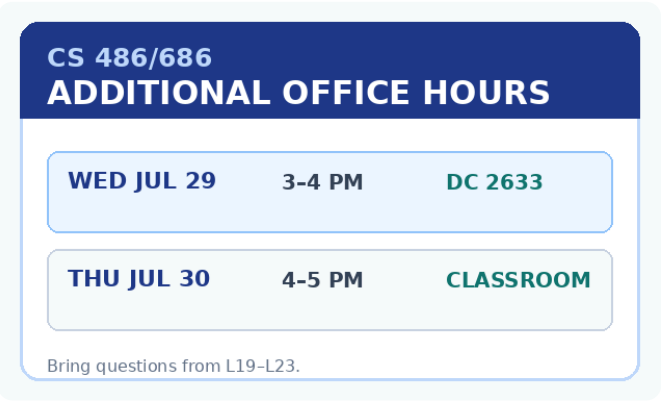
generate
emit text tokens

Adapted from the official Transformers.js Qwen3.5 WebGPU example.

Read a document with the same model LIVE IN BROWSER

The model loaded for VQA is reused—choose the announcement or upload a document.

Slow-device warning: live generation can take around 20 minutes. The recorded result is immediate, and you can stop live generation at any time.



RECORDED QWEN3.5-0.8B OUTPUT

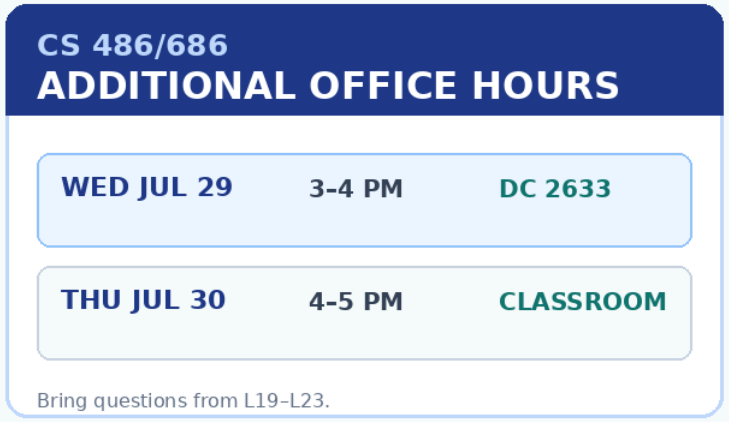
QUESTION

Where and when are the Wednesday office hours?

ANSWER

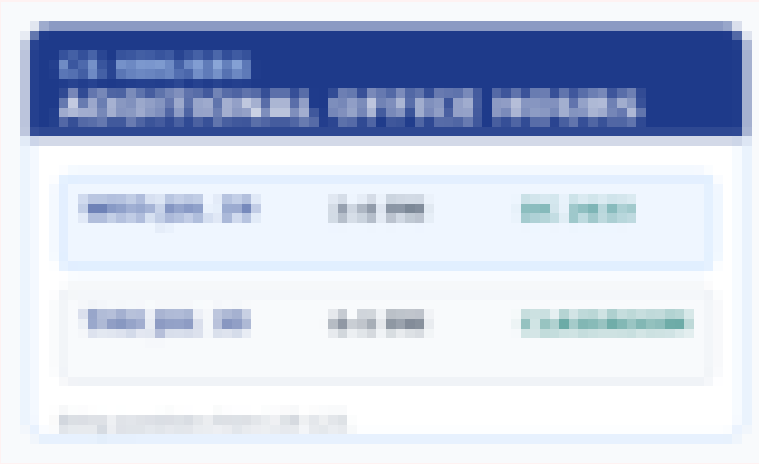
Wednesday 3-4 PM at DC 2633

Too few patches can erase the evidence



240 embeddings

Qwen: “Wednesday 3-4 PM at DC 2633”



77 embeddings

Qwen: “Wednesday 3:00 PM” — room lost

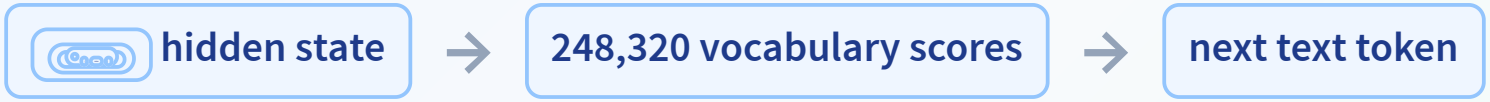
The model cannot recover visual evidence that preprocessing removed.

Recap: adapt the image, then reuse the LLM

1	**Patch**	the image into 16×16 regions.
2	**Encode**	them as 768-wide semantic features.
3	**Merge + project**	2×2 groups into 1024-wide image embeddings.
4	**Concatenate**	image and word embeddings through the same interface.
5	**Reuse Qwen**	to predict the text answer.

LOOK AT THE OUTPUT HEAD

Qwen can see images—but it can only emit text tokens.



No image decoder. No denoiser. No pixels out.

Next: how do models generate images and future frames?

L24: Dissecting Diffusion and World Models.